EF01 Task

🧪 Lab Assignment

Mapping Classes to Database using Entity Framework Core (Code First)

📘 Scenario

You are building a simple **Bookstore System**.

Your objective is to map C# classes to a database using the **Entity Framework Core – Code First approach**.

During this lab, you will practice:

- Creating entity classes
- Configuring a `DbContext`
- Running migrations
- Applying three different EF Core mapping strategies

⚙️ Project Setup

Step 1 — Create the Project

- Open **Visual Studio 2022**
- Create a new **Console Application**
- Target framework: **.NET 5**

Step 2 — Install Required Packages

Open **NuGet Package Manager** and install:

- `Microsoft.EntityFrameworkCore.SqlServer` (version 5.x)

- `Microsoft.EntityFrameworkCore.Tools` (version 5.x)

> ⚠️ Important: The EF Core package version must match your .NET version.

🧩 Task 1 — Create Entity Classes

Create a folder named:

```
Models
```

Inside it, create the following classes:

📗 Class: `Book`

Property	Type	Requirements
Id	int	Primary Key
Title	string	Required, Max Length 150
Price	decimal	Required
PublishedDate	DateTime?	Optional

📘 Class: `Author`

Property	Type	Requirements
Id	int	Primary Key
Name	string	Required, Max Length 100
Country	string?	Optional

Task 2 — Create the DbContext

Create a folder named:

```
DbContexts or Contexts
```

Inside it, create a class named:

```
BookstoreDbContext
```

Requirements:

1. Inherit from `DbContext`
2. Override `OnConfiguring`
3. Use the following connection string:

```
Server=.;Database=BookstoreDB;Trusted_Connection=True;Trust
ServerCertificate=True;
```

1. Define a `DbSet<T>` for each entity class.

Task 3 — Run Your First Migration

Open **Package Manager Console** and run:

```
Add-Migration InitialCreate
Update-Database
```

After Execution:

- Open the generated migration file
- Read the `Up()` and `Down()` methods
- Open:
  - SQL Server Management Studio

    OR
  - SQL Server Object Explorer in Visual Studio
- Take a screenshot showing:
  - The created database
  - The generated tables

Task 4 — Apply Three Mapping Strategies

Part A — Mapping by Convention

Without adding any attributes or Fluent API:

At the top of `Book.cs`, answer the following questions as comments:

```
Q1: Why did the property "Id" become a Primary Key
    without any explicit configuration?

Q2: Why is "Country" nullable in the database
    while "Price" is not?
```

Part B — Data Annotations

Open the `Author` class and apply:

- `[Required]` to `Name`
- `[MaxLength(100)]` to `Name`
- `[Table("Writers")]` to map the class to a table named **Writers**

Then run:

```
Add-Migration AnnotateAuthor
Update-Database
```

Part C — Fluent API - in a clean way - Configuration Class Per Entity

In `BookstoreDbContext`, override:

```
OnModelCreating()
```

Configure the `Book` entity:

- `Title` → Required, Max Length 150
- `Price` → Column Type `decimal(8,2)`

- `PublishedDate` → Explicitly Optional

Then run:

```
Add-Migration FluentBook
Update-Database
```

---

follow this Modularity

```
BookstoreSystem/
│
├── Models/
│   ├── Book.cs
│   └── Author.cs
│
├── Contexts/
│   └── BookstoreDbContext.cs
│
├── Migrations/
│   └── (All migration files)
│
└── Screenshots/
    └── database_tables.png
```

Important Rule

Your C# code is the **only source of truth**.

Do NOT create or modify tables directly in SQL Server.

All database changes must be applied using:

```
Migrations
```

why we Use Code first rather than DBfirst?
Why we use Fluent API ovber than Data Annotations? (in PDF)